



Tsinghua University

Modern Computer Architecture

Fall 2025

Lecture 07_2 Cache Intro

Yongpan Liu

ypliu@tsinghua.edu.cn

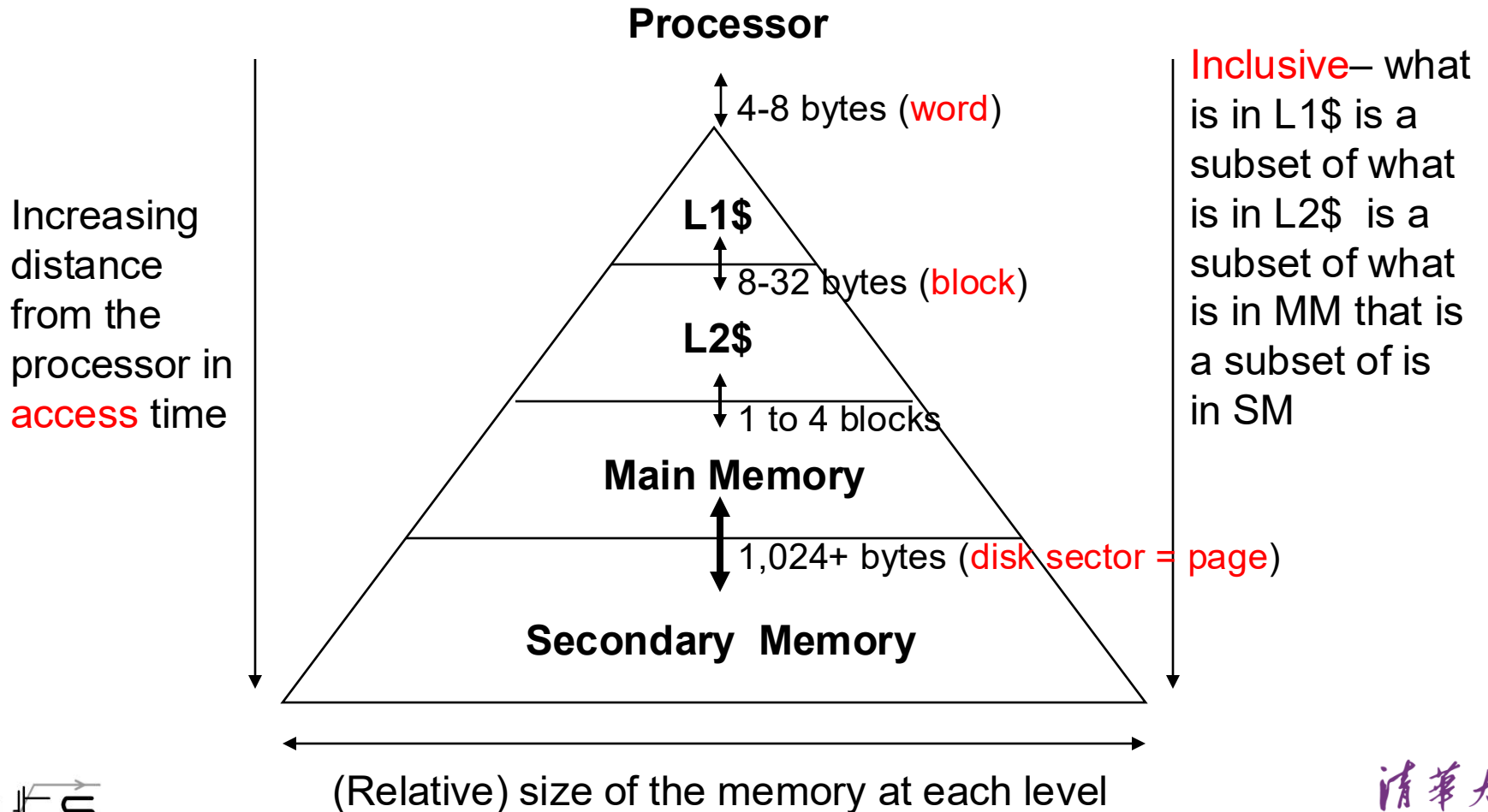
2025-Nov.-12

Outline of Today

- Memory Overview
- Memory Hierarchy
- Cache Introduction

Review: The Memory Hierarchy

- Take advantage of **the principle of locality** to present the user with as **much** memory as is available in the **cheapest** technology at the speed offered by the **fastest** technology



How is the Hierarchy Managed?

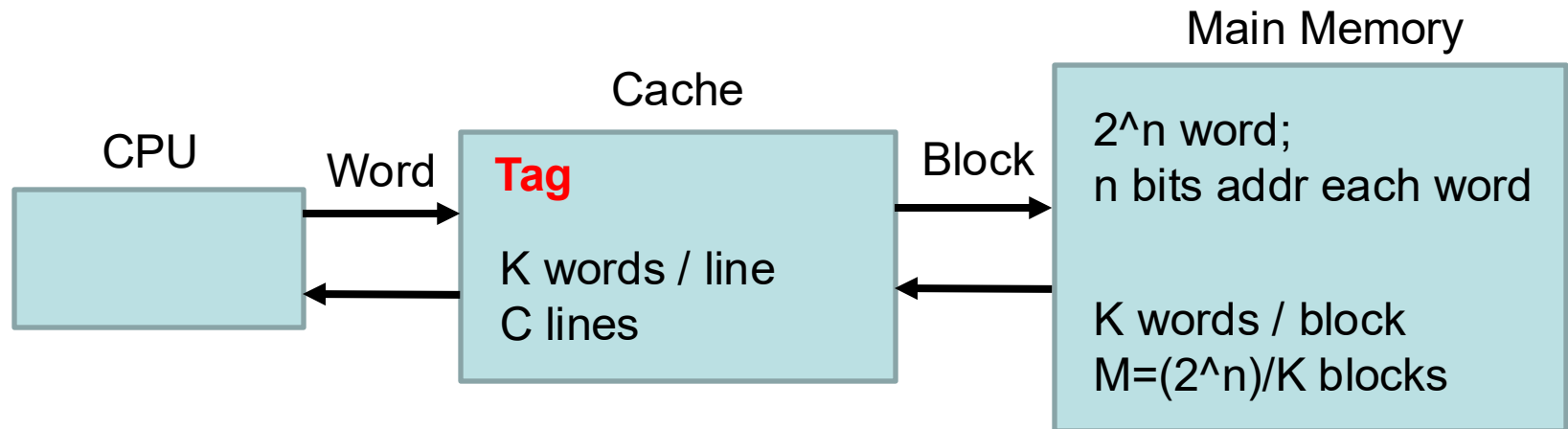
- registers $\leftrightarrow$ memory
 - by compiler (programmer?)
- cache $\leftrightarrow$ main memory
 - by the cache controller hardware
- main memory $\leftrightarrow$ disks
 - by the operating system (virtual memory)
 - virtual to physical address mapping assisted by the hardware (TLB)
 - by the programmer (files)

Outline of Cache Intro.

- Simple Cache Example
- Handling Cache Hits
- Handling Cache Miss

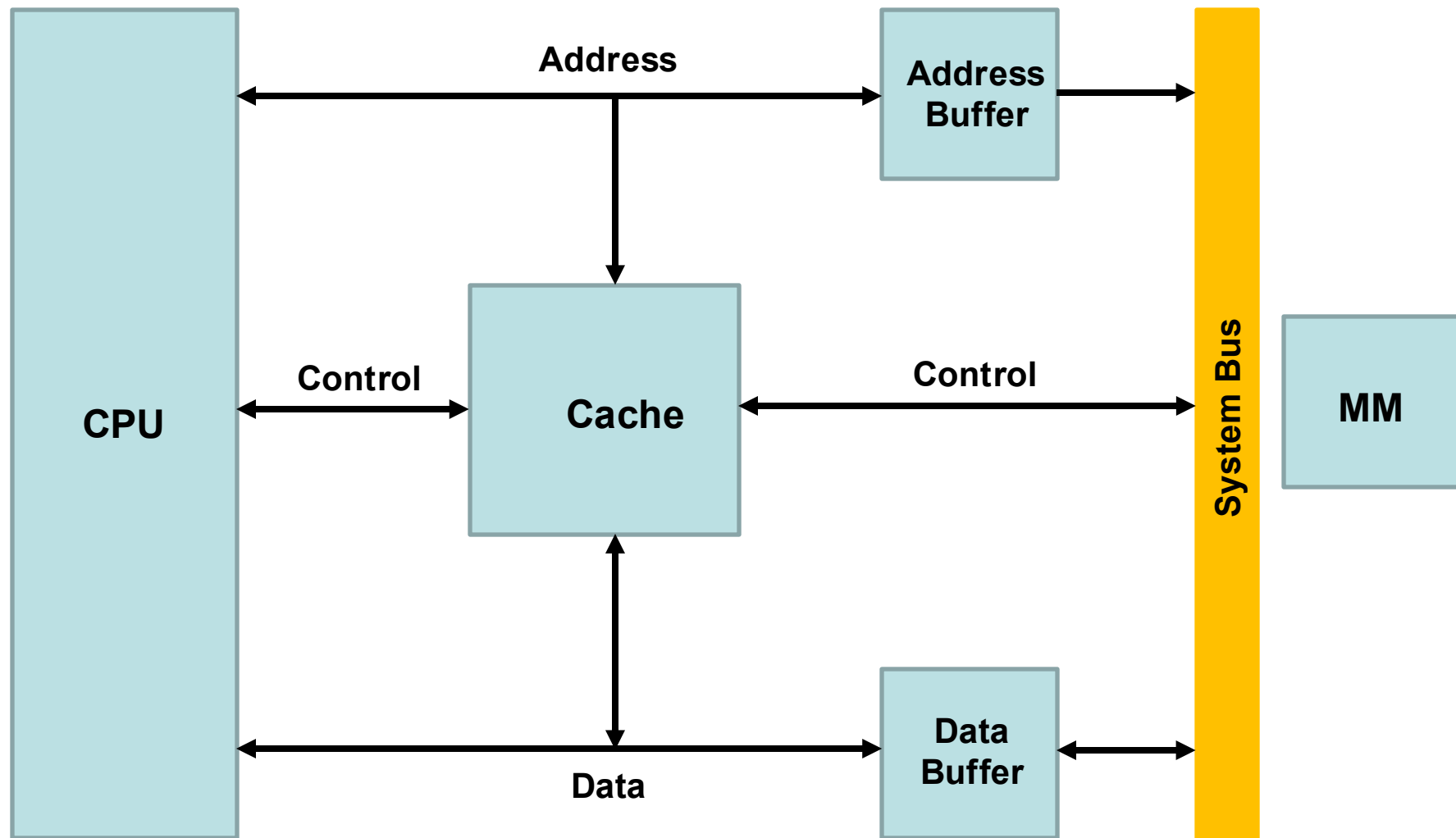
Cache

- 3 questions to answer (in hardware):
 - Q1: How do we know if a data item is in the cache?
 - Q2: If it is, how do we find it?
 - Q3: 如果数据不在cache中，该怎么办？



Tag : parts of Main Memory address

Typical Cache Organization



Caching: A Simple First Example

- Direct mapped
 - For each item of data at the lower level, there is exactly one location in the cache where it might be - so lots of items at the lower level must **share** locations in the upper level
 - Address mapping:

(MM block address) modulo (# of blocks in the cache)

- First consider block sizes of **one word**

Caching: A Simple First Example

Valid bit indicates whether an entry contains valid information – if the bit is not set, there cannot be a match for this block

Cache			
Index	Valid	Tag	Data
00			
01			
10			
11			

Q1: Is it there?

Compare the cache tag to the high order 2 memory address bits to tell if the memory block is in the cache

Main Memory

	0000xx
	0001xx
	0010xx
	0011xx
	0100xx
	0101xx
	0110xx
	0111xx
	1000xx
	1001xx
	1010xx
	1011xx
	1100xx
	1101xx
	1110xx
	1111xx

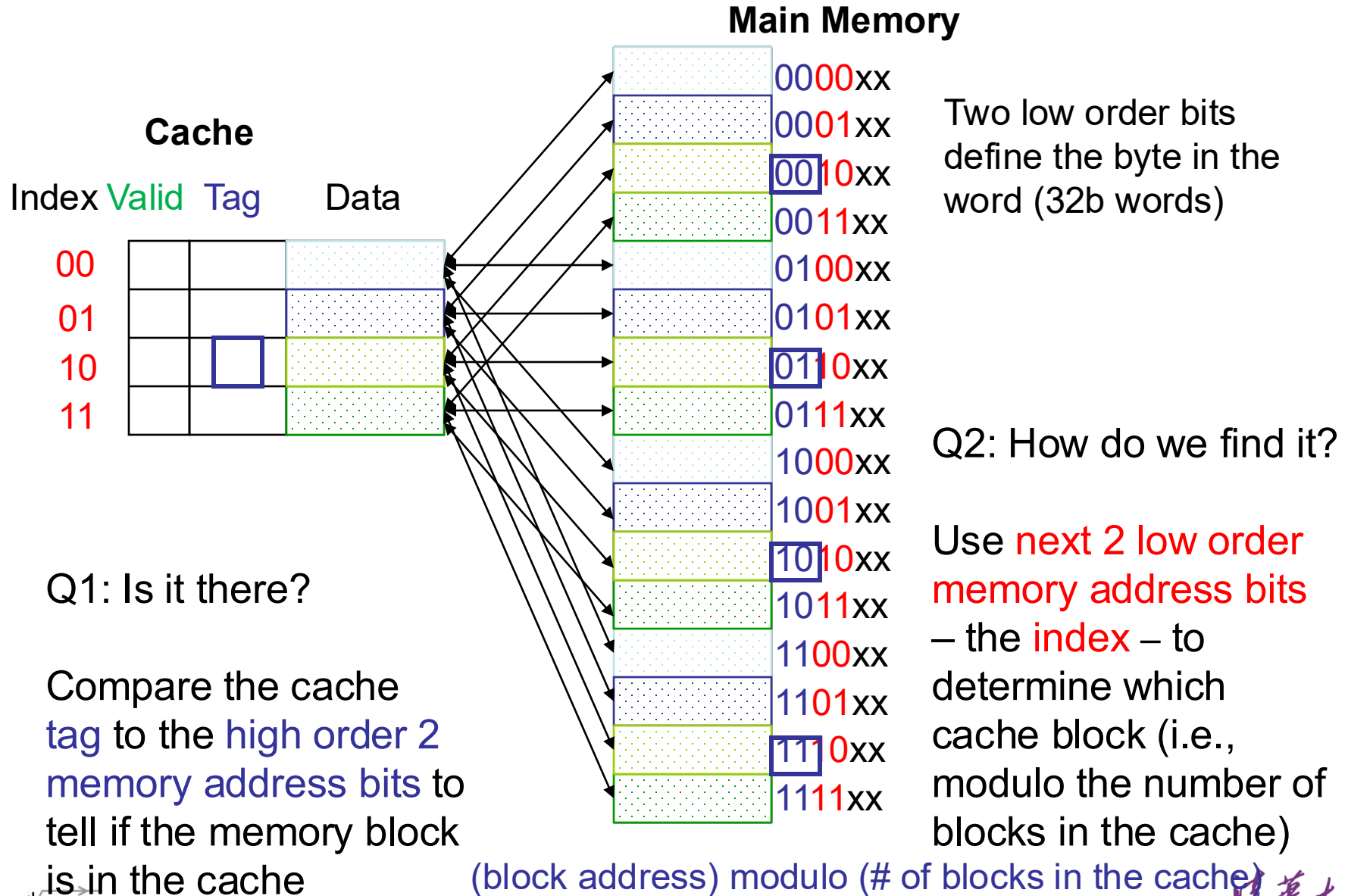
Two low order bits define the byte in the word (32b words)

Q2: How do we find it?

Use next 2 low order memory address bits – the index – to determine which cache block (i.e., modulo the number of blocks in the cache)

(block address) modulo (# of blocks in the cache)

A4: Caching: A Simple First Example



Direct Mapped Cache

- Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 1 2 3 4 3 4 15
0000 0100 1111 (tag:index)

Index	Tag	Data
-------	-----	------

0 miss

00	00	Mem(0)
01		
10		
11		

1 miss

00	Mem(0)
00	Mem(1)

2 miss

00	Mem(0)
00	Mem(1)
00	Mem(2)

3 miss

00	Mem(0)
00	Mem(1)
00	Mem(2)
00	Mem(3)

4 miss

01	00	Mem(0)
	00	Mem(1)
	00	Mem(2)
	00	Mem(3)

3 hit

01	Mem(4)
00	Mem(1)
00	Mem(2)
00	Mem(3)

4 hit

01	Mem(4)
00	Mem(1)
00	Mem(2)
00	Mem(3)

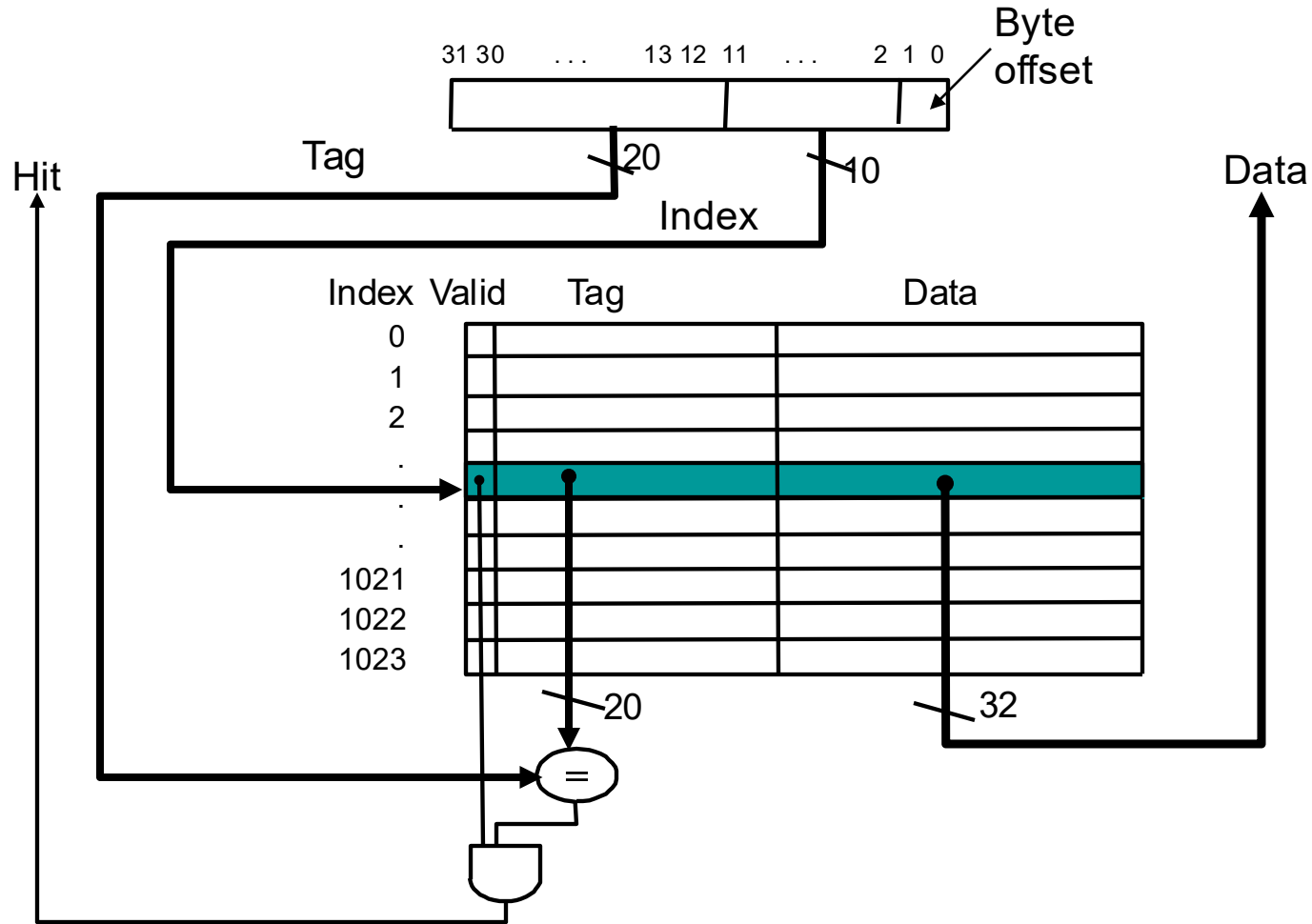
15 miss

	01	Mem(4)
	00	Mem(1)
	00	Mem(2)
11	00	Mem(3)

□ 8 requests, 6 misses

MIPS Direct Mapped Cache Example

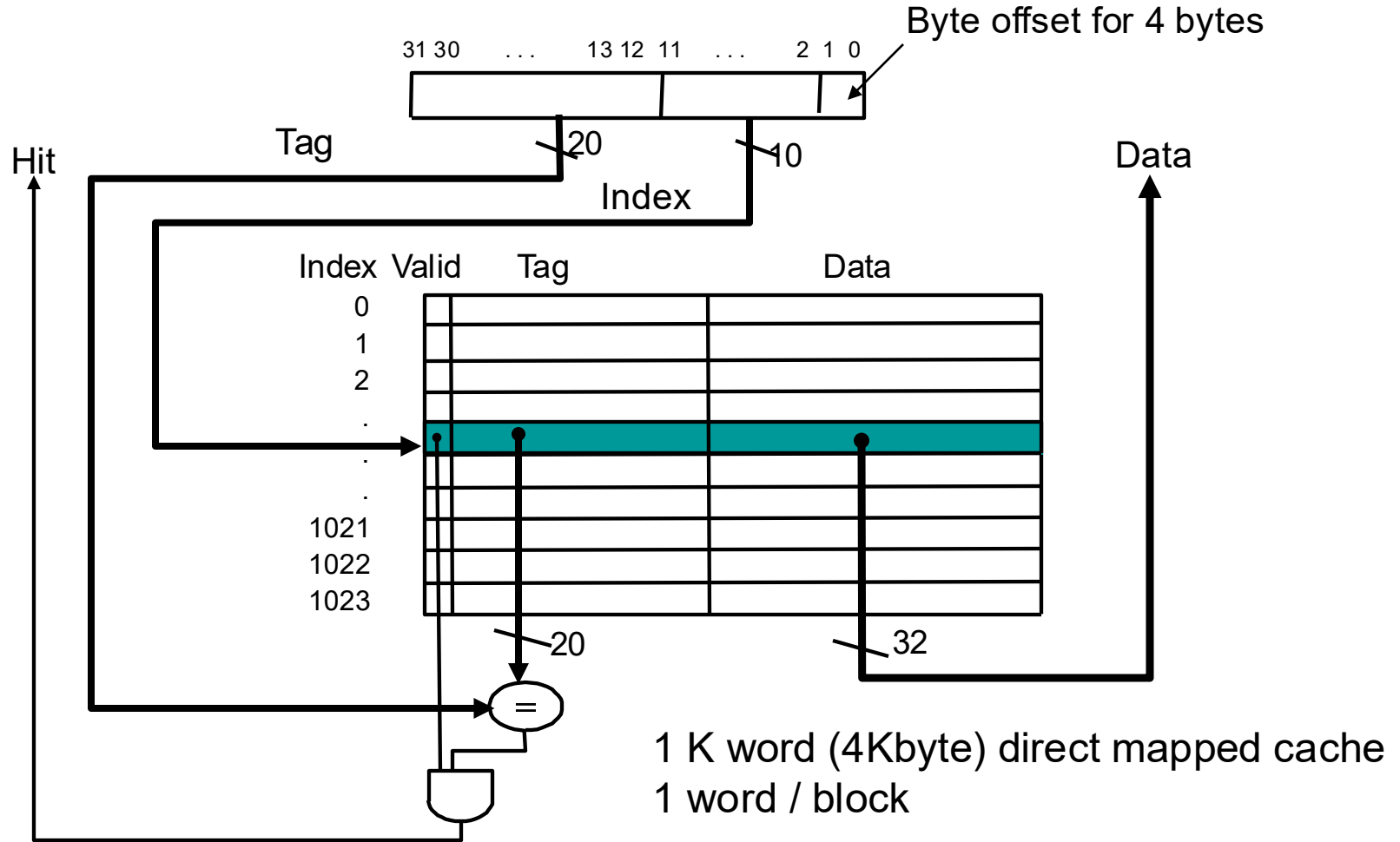
- One word/block, cache size = 1K words



What kind of locality are we taking advantage of?

MIPS Direct Mapped Cache Example

- One word/block, cache size = 1K words

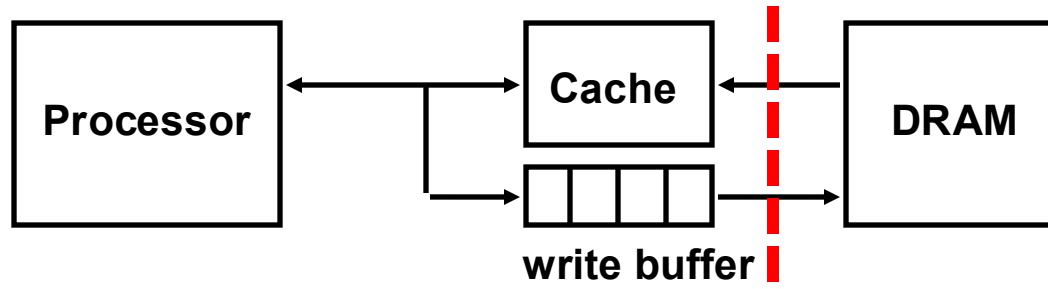


What kind of locality are we taking advantage of?
Temporal!

Handling Cache Hits

- Read hits (I\$ and D\$)
 - this is what we want!
- Write hits (D\$ only)
 - allow cache and memory to be **inconsistent**
 - **write-back** : write the data only into the cache block
 - need a **dirty** bit for each data cache block to tell if it needs to be written back to memory when it is evicted
 - require the cache and memory to be **consistent**
 - **write-through**: always write the data into both
 - writes run at the speed of the next level in the memory hierarchy – so slow! – or can use a **write buffer**, so only have to stall if the write buffer is full

Write Buffer for Write-Through Caching



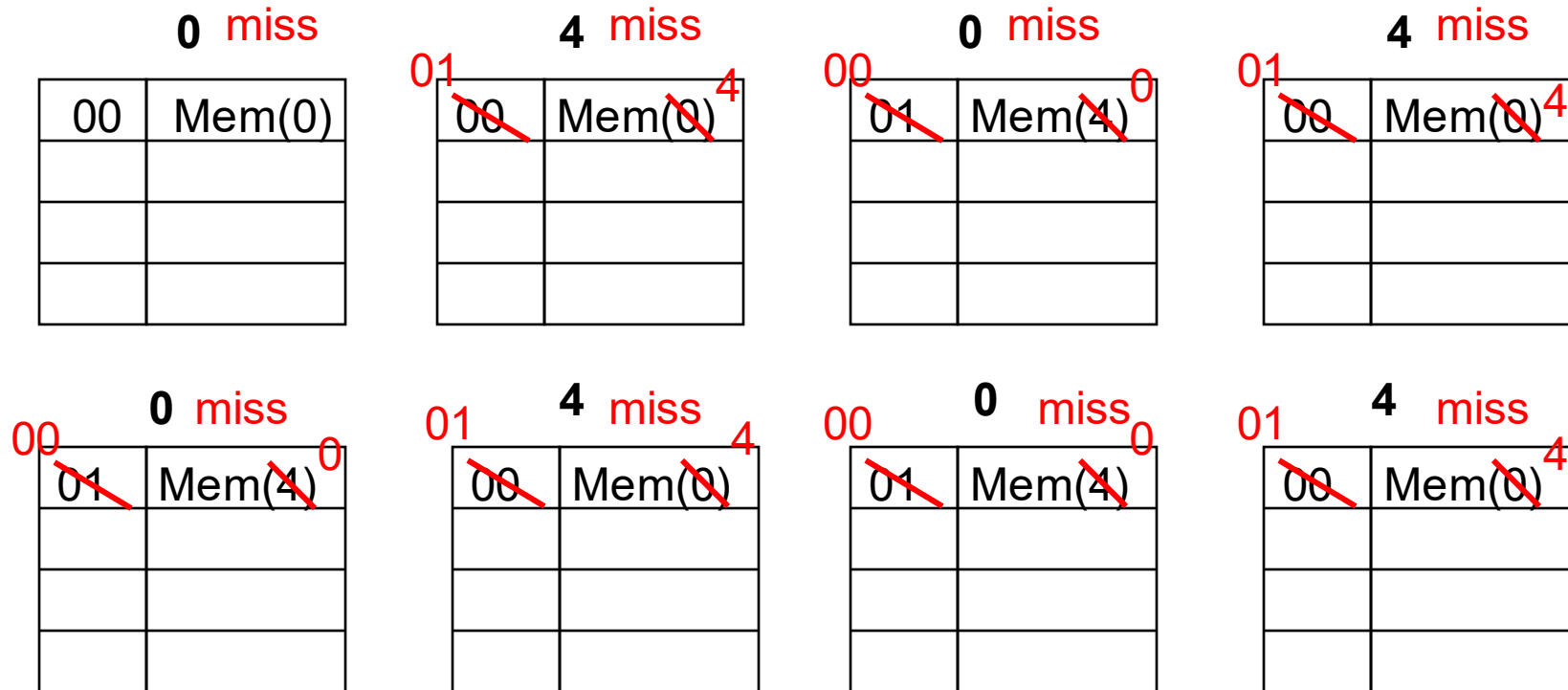
- Write buffer between the cache and main memory
 - Processor: writes data into the cache and the write buffer
 - Memory controller: writes contents of the write buffer to memory
- The write buffer is just a FIFO
 - Typical number of entries: 4
 - Works fine if $\text{store frequency (w.r.t. time)} \ll 1 / \text{DRAM write cycle}$
- Memory system designer's nightmare
 - When the $\text{store frequency (w.r.t. time)} \rightarrow 1 / \text{DRAM write cycle}$ leading to write buffer **saturation** (the CPU must stop and wait) .
 - use a **write-back cache**;
 - use **an L2 cache** (shorter write time than DRAM or 2nd level write back, next lecture)

Another Reference String Mapping

- Consider the main memory word reference string

Start with an empty cache - all
blocks initially marked as not valid

0 4 0 4 0 4 0 4



□ 8 requests, 8 misses

- **Ping pong effect** due to **conflict** misses - two memory locations that map into the same cache block

3C: Sources of Cache Misses

- **Compulsory** (cold-start or first reference):
 - First access to a block
 - When we first start the program...
 - compulsory misses usually don't bother you
- **Conflict** (collision):
 - Multiple memory locations mapped to the same cache location (e.g. Ping pong effect)
 - Solution 1: increase cache size
 - Solution 2: increase associativity (next lecture)
- **Capacity**:
 - Cache cannot contain all blocks accessed by the program
 - Solution: increase cache size

Handling Cache Misses

- Read misses (I\$ and D\$)
 - **stall** the pipeline, fetch the block from the next level in the memory hierarchy,
 - put it in the cache (which may first require evicting a dirty block if using a write-back cache) (it's now clean)
 - send the requested word to the processor, let the pipeline resume
- Write misses (D\$ only)
 1. **stall** the pipeline, fetch the block from the next level in the memory hierarchy, put it in the cache (which may first require evicting a dirty block if using a write-back cache) (it's now clean), write the word from the processor to the cache (it's now dirty), then let the pipeline resume
 2. **Write allocate** – just write the word into the cache, updating both the tag and data – **no need to stall**
 3. **No-write allocate** – skip the cache write and just write the word to the write buffer (and eventually to the next memory level) – **no need to stall** if the write buffer isn't full

Direct Mapped Cache

- Consider the main memory word reference string

Start with an empty cache - all blocks initially marked as not valid

0 1 2 3 4 3 4 15
0000 0100 1111

0 miss

00	Mem(0)

1 miss

00	Mem(0)
00	Mem(1)

2 miss

00	Mem(0)
00	Mem(1)
00	Mem(2)

3 miss

00	Mem(0)
00	Mem(1)
00	Mem(2)
00	Mem(3)

4 miss

01

00	Mem(0)
00	Mem(1)
00	Mem(2)
00	Mem(3)

4

3 hit

01	Mem(4)
00	Mem(1)
00	Mem(2)
00	Mem(3)

4 hit

01	Mem(4)
00	Mem(1)
00	Mem(2)
00	Mem(3)

15 miss

11

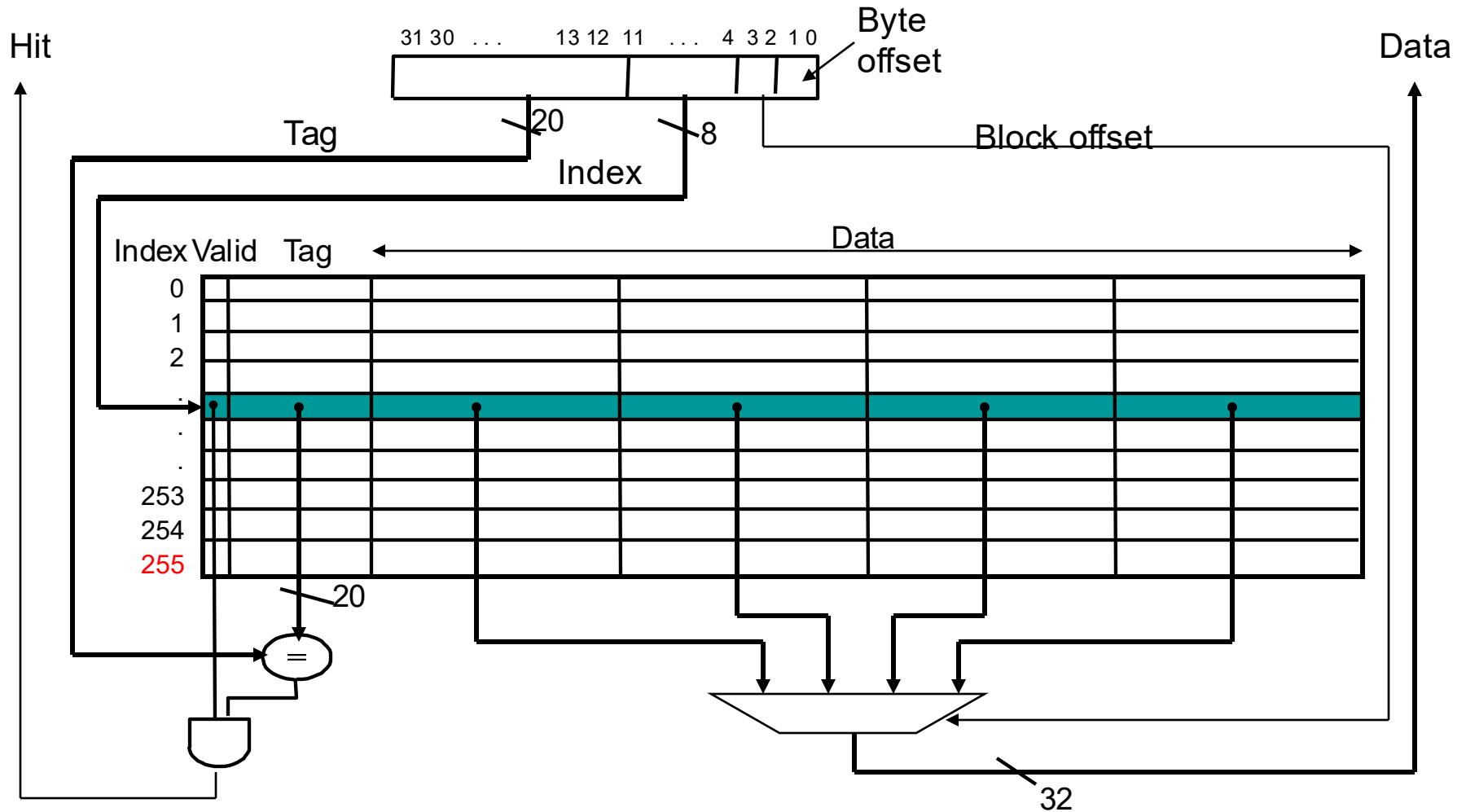
01	Mem(4)
00	Mem(1)
00	Mem(2)
00	Mem(3)

15

□ 8 requests, 6 misses

Multiword Block Direct Mapped Cache

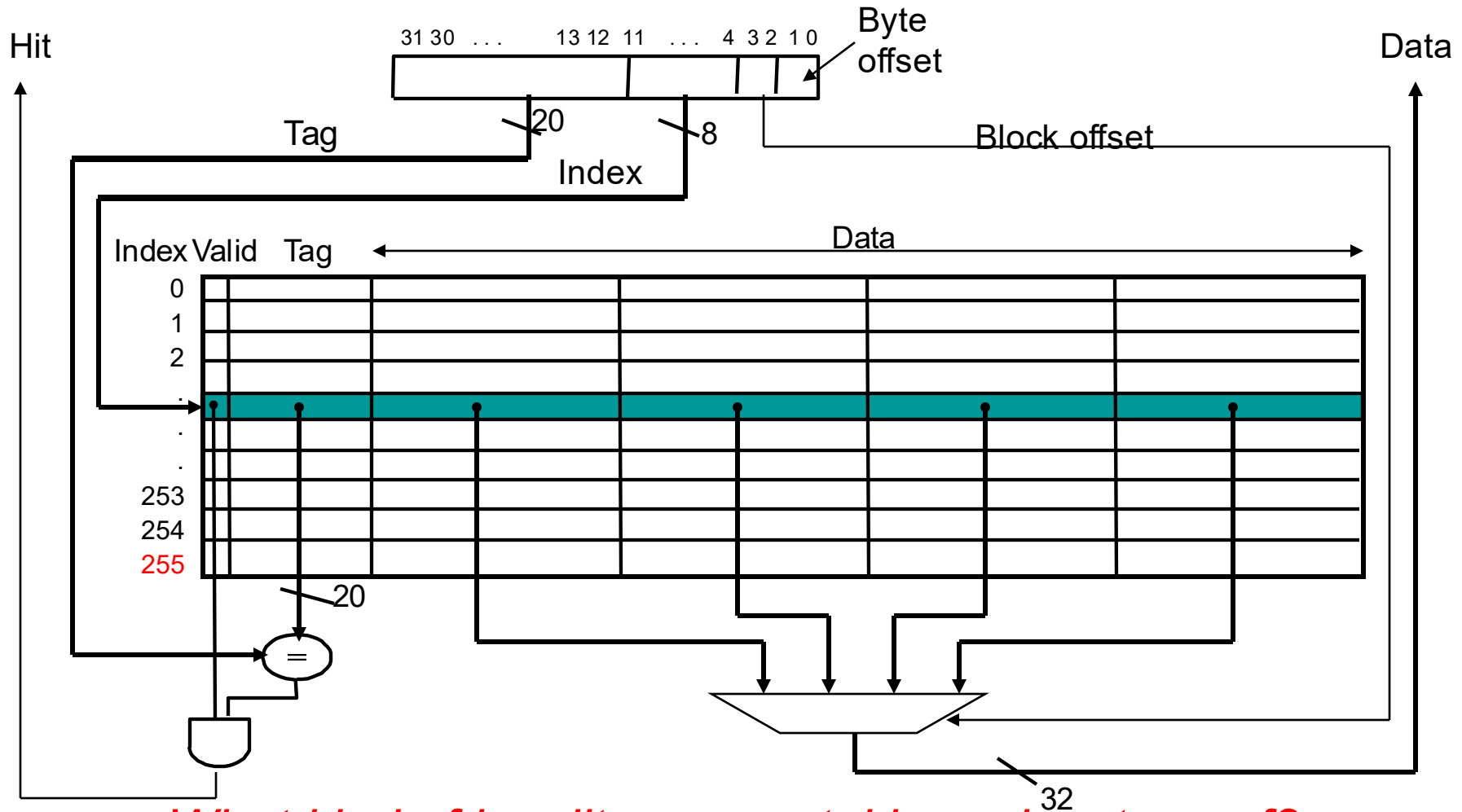
- Four words/block, cache size = 1K words



What kind of locality are we taking advantage of?

Multiword Block Direct Mapped Cache

- Four words/block, cache size = 1K words



What kind of locality are we taking advantage of?

Spatial ! + Temporal!

Taking Advantage of Spatial Locality

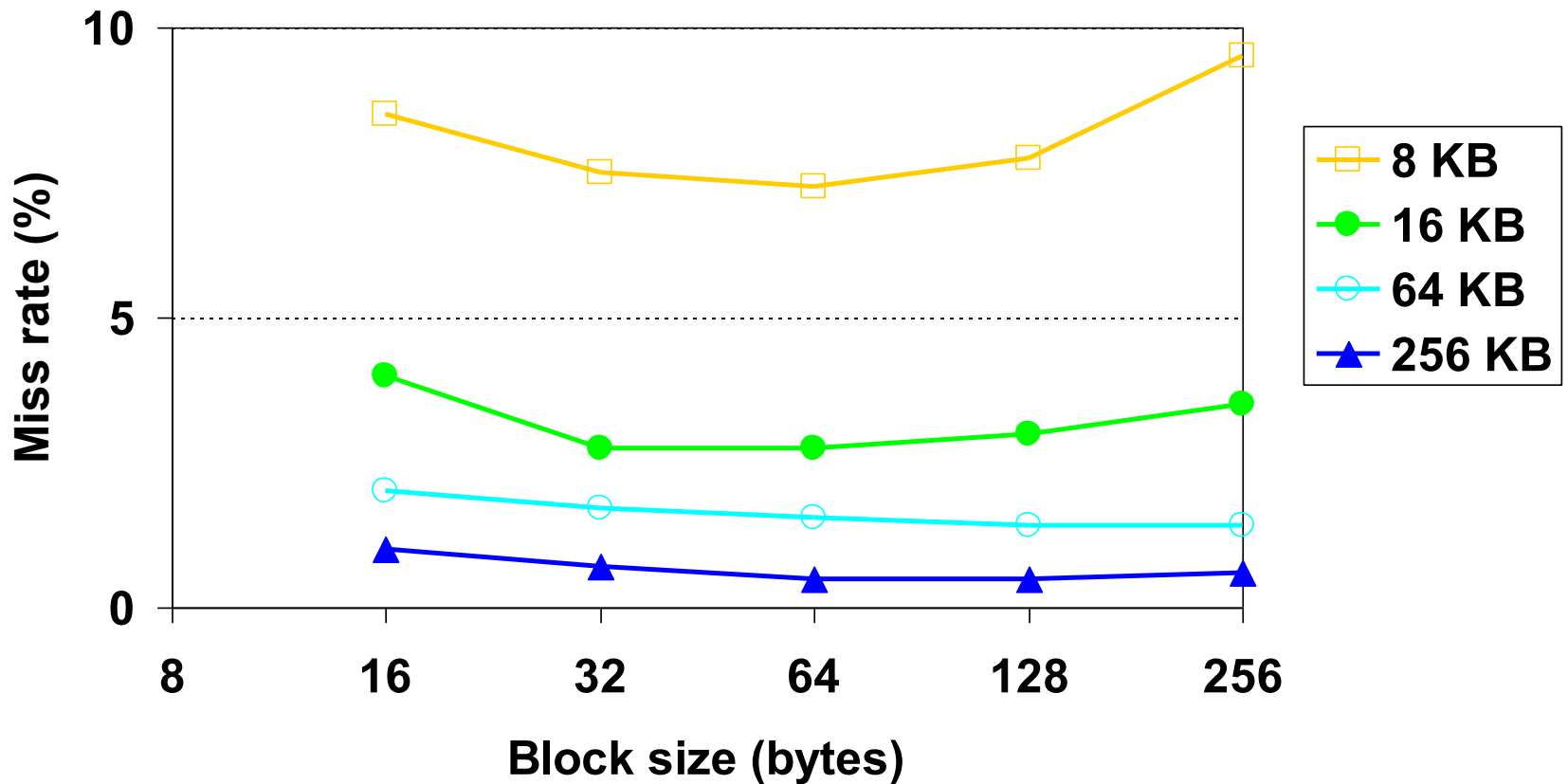
- Let cache block hold 2 words

Index	Tag	Data:1	Data:0	0	1	2	3	4	3	4	15	
				0000			0010			Tag(2): index(1): block offset(1)		
				0 miss			1 hit			2 miss		
0	00	Mem(1)	Mem(0)	00	Mem(1)	Mem(0)	00	Mem(1)	Mem(0)	00	Mem(1)	Mem(0)
1										00	Mem(3)	Mem(2)
				3 hit			4 miss			3 hit		
	00	Mem(1)	Mem(0)	01	Mem(1)	Mem(0)	01	Mem(5)	Mem(4)	01	Mem(5)	Mem(4)
	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)
				4 hit			15 miss					
	01	Mem(5)	Mem(4)	11	Mem(5)	Mem(4)	11	Mem(3)	Mem(2)	01	Mem(5)	Mem(4)
	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)	00	Mem(3)	Mem(2)

□ 8 requests, **4 misses**

Start with an empty cache - all blocks initially marked as not valid

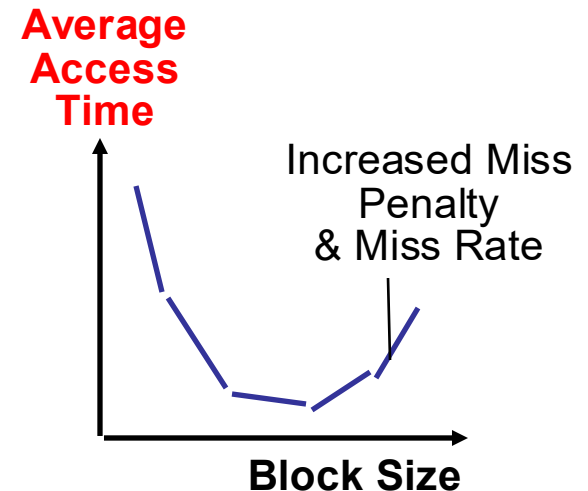
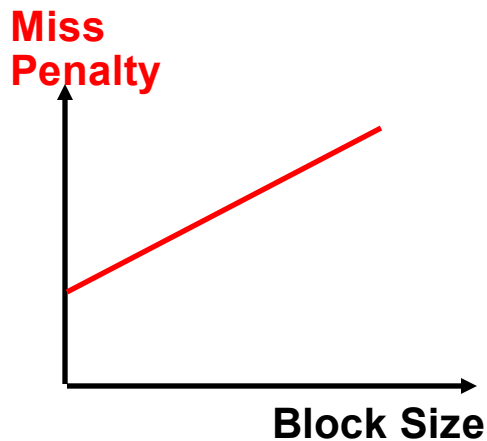
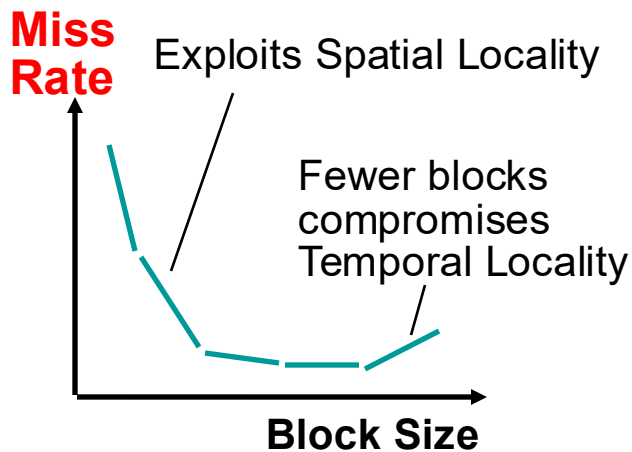
Miss Rate vs Block Size vs Cache Size



- Miss rate goes up if the block size becomes a significant fraction of the cache size, because the number of blocks that can be held in the same size cache is smaller (increasing **capacity** misses)

Block Size Tradeoff

- Larger block sizes take advantage of spatial locality **but**
 - If the block size is too big relative to the cache size, the miss rate will go up (for Capacity Miss)
- Larger block size means larger miss penalty
 - Latency to first word in block + transfer time for remaining words



□ In general, **Average Memory Access Time**
= Hit Time + Miss Penalty x Miss Rate

Multiword Block Considerations

- Read misses (I\$ and D\$)
 - Processed the same as for single word blocks – a miss returns the entire block from memory
 - Miss penalty grows as block size grows
 - **Early restart** – datapath resumes execution as soon as the requested word of the block is returned
 - **Requested word first** – requested word is transferred from the memory to the cache (and datapath) first
 - **Nonblocking cache** – allows the datapath to continue to access the cache while the cache is handling an earlier miss
- Write misses (D\$)
 - **Can't use write allocate** or will end up with a “garbled” block in the cache (e.g., for 4 word blocks, a new tag, one word of data from the new block, and three words of data from the old block), so must fetch the block from memory first and pay the stall time

Cache Summary

- The Principle of Locality:
 - Program likely to access a relatively small portion of the address space at any instant of time
 - Temporal Locality: Locality in Time
 - Spatial Locality: Locality in Space
- Three major categories of cache misses:
 - Compulsory misses: sad facts of life. Example: cold start misses
 - Conflict misses: increase cache size and/or associativity
Nightmare Scenario: ping pong effect!
 - Capacity misses: increase cache size
- Cache design space
 - total size, block size, associativity (replacement policy)
 - write-hit policy (write-through, write-back)
 - write-miss policy (write allocate, write buffers)

Today's Reading Assignments

- Computer Organization and Design: The Hardware/Software Interface, ARM版, 4rd
 - CH 5.1 ~ 5.3
 - P535, Ex5.2-5.3